AI Infrastructure Resource Ledger: The Reality Check

Thursday, 5. March 2026

Engineer(s): Zack Allen

Date: 3/5/26

Project Title: VLA Overseer and Corrective Action System

Phase 1: The Calendar (Working Backward)

If you are still writing your training loop on your Drop-Dead Date, your project will fail.

1. Final Demo / Presentation Date: 4/21/26
2. Days required for Evaluation & Report: 7 days.
3. Estimated Training Run Duration: 14 days.
4. DROP-DEAD TRAINING START DATE: 3/30/26 *(Subtract Line 2 & 3 from Line 1)*

Phase 2: The Stack & Compute Target

1. Base architecture backbone, include encoders etc., add as many lines as you need:

   Octo Small 27M Watcher, OpenVLA 7B Corrector
   Param Count: 27M and 7B

2. Target Compute Hardware:
   [X] Local / Lab Node: Thor
   [X] Cloud Rental (e.g., Lambda/AWS):
3. Specific GPU(s): H100? Thor
   Total VRAM Available: 80 GB, 100 GB

Phase 3: The Memory Math (The VRAM Ledger)

Reference: 1B Parameters ≈ 2GB (Weights) + 2GB (Gradients) + 12GB (AdamW Optimizer States)

Over Watcher

1. Model Weights (BF16): 0.06 GB
2. Gradients (BF16): 0.005 GB
3. Optimizer States: 0.03 GB
4. KV Cache & Activations Reserve: 1 GB
5. TOTAL REQUIRED VRAM: 1.1 GB

VLA (when activated)

6. Model Weights (BF16): 14.1 GB
7. Gradients (BF16): 0.14 GB
8. Optimizer States: 0.84 GB

9. KV Cache & Activations Reserve: 18 GB
10. TOTAL REQUIRED VRAM: 33 GB

Recovery Generator Head (when activated)

11. Model Weights (BF16): 14 GB
12. Gradients (BF16): 0.1 GB
13. Optimizer States: 0.86 GB
14. KV Cache & Activations Reserve: 4 GB
15. TOTAL REQUIRED VRAM: 18 GB

TOTAL REQUIRED VRAM: 34.1 GB Peak

The Strategy: Is your Total Required VRAM > Total Available VRAM?

[ ] Yes [X] No

If Yes, what is your exact mitigation strategy? [ ] FSDP / ZeRO-3 (Sharding across multiple GPUs)

[ ] LoRA / QLoRA (Freezing base weights, avoiding optimizer state explosion)

[ ] I am shrinking my model selection.

[ ] Something else (explain):

Phase 4: The Data Ingress (The I/O Bottleneck)

1. Dataset Size: 1 GB (real) and 20 GB OpenX-embodiment, GB / 11,500 Total Samples

2. Format:
   [] Raw JPEGs/PNGs/trajectories/etc. in a folder (High I/O Risk)
   [X] WebDataset / Sharded .tar
   [ ] Something else

3. Storage Location:
   [X] Local NVMe SSD
   [ ] Network Drive
   [ ] S3/GCS Bucket
   [ ] Something else

If using raw images or other large data quantities over a network or standard SSD, how will you prevent your dataloader from starving your GPUs?

Phase 5: Red Team Audit (Partner Review)

Reviewing Engineer: Mel Krusniak

Look at your partner's ledger. Find the fatal flaw. Are they underestimating training time? Do they have a 40GB model fitting into 24GB of VRAM? Are they trying to load 2 million JPEGs sequentially?

The Fatal Flaw / Biggest Risk:

I have questions about how this recovery generation head is going to work. What constitutes a “recovery”? Seemingly it’s not just an action series but a modification to the underlying FSM / decision tree / classical model. It’s very unclear to me how this will be trained. But if you were to skip it and just use actions straight out of OpenVLA, it’s (1) unclear how you’d determine that recovery is finished, and (2)

Phase 6: The 72-Hour Contract

What exact, granular engineering task will you complete in the next 72 hours to unblock these risks? (e.g., "Write dummy dataloader script to verify throughput," "Run a 10-step overfitting batch to check VRAM usage.")

Deliverable by Monday: I am planning on generating real world data to full define the input to the VLA. I am hoping to generate 200 samples and label them and hopefully use another model that will be able to automatically label the samples generated after that. I will begin the architecture for the observer / fault detector Octo model with training. This will give me the necessary inputs to pass into OpenVLA to start looking at some of the outputs from it. (talked in front of class: need additional data (supervisory signal) to perform supervised fine tuning